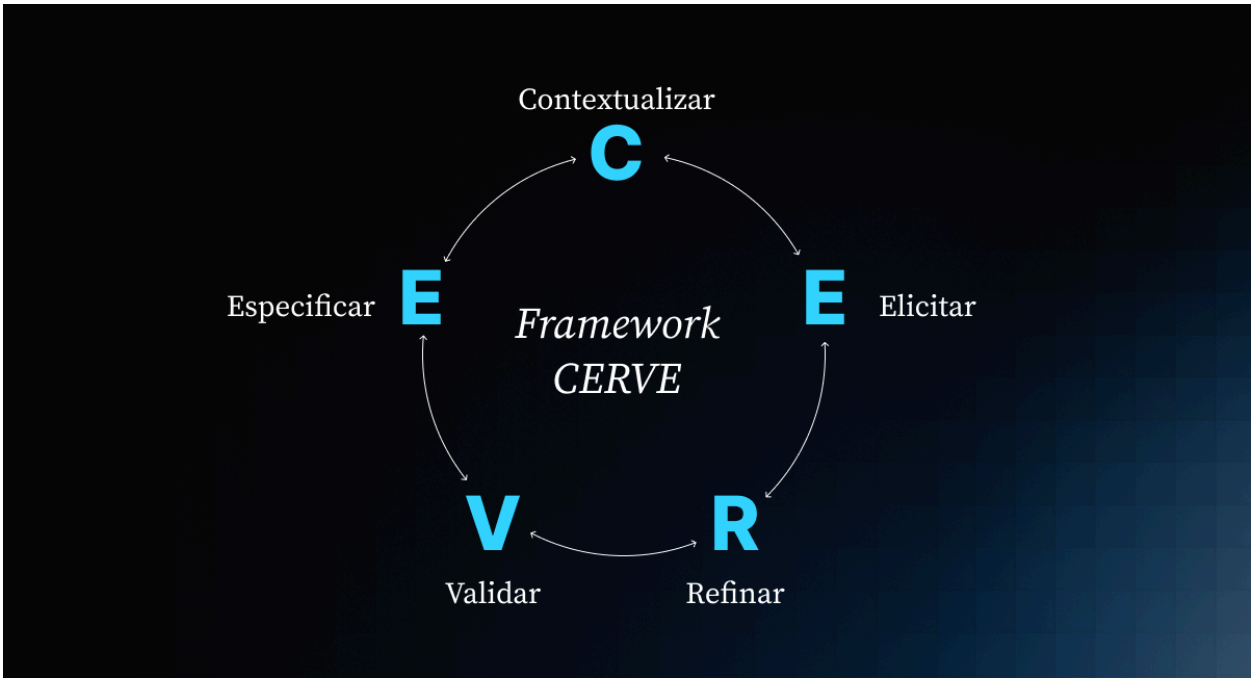
Framework CERVE

Framework CERVE



Você tem o vocabulário certo. Sabe o que são regras de negócio, requisitos, histórias e critérios de aceite. Sabe o que documentar e o quanto documentar.

O que falta agora é um processo. Uma sequência que organize o trabalho de especificação do começo ao fim, sem deixar decisão importante cair no meio do caminho.

O Framework CERVE é isso. Cinco etapas que levam o projeto desde a compreensão do problema até o backlog pronto para desenvolvimento. O nome é um acrônimo: **C**ontextualizar, **E**licitar, **R**efinar, **V**alidar e **E**specificar.

Para ilustrar cada etapa, vamos usar o mesmo exemplo ao longo de toda a seção: o desenvolvimento de um módulo de emissão de segunda via de boleto para uma plataforma educacional.

C — Contextualizar

Entender profundamente o problema antes de qualquer solução.

Essa é a etapa mais ignorada e a mais cara de ignorar. Quando um projeto começa pela solução, tudo que vier depois estará tentando acertar um alvo que ainda não foi bem definido. Contextualizar é construir a base: o porquê do projeto, o que está em jogo, quais são os limites e o que já se sabe antes de conversar com qualquer stakeholder.

Ferramentas e atividades:

- **Entrevistas com stakeholders:** conversas estruturadas para entender objetivos, restrições e expectativas de quem tem autoridade sobre o negócio
- **Briefing do projeto:** documento inicial que registra escopo, contexto, prazo e principais envolvidos
- **Análise de documentação existente:** leitura de materiais já produzidos sobre o domínio: processos, contratos, manuais, sistemas legados
- **Workshops de alinhamento:** sessões colaborativas para construir entendimento compartilhado entre áreas diferentes

Resultado esperado:

- Objetivos de negócio claros
- Problema definido com precisão
- Lista de Certezas, Suposições e Dúvidas

Exemplo: a plataforma recebe um volume alto de chamados no suporte sobre boletos vencidos. O objetivo de negócio é reduzir esse volume em 40% nos próximos dois trimestres. Antes de qualquer reunião de requisitos, o time registra o que já sabe (certeza: os boletos expiram em 3 dias úteis), o que está supondo (suposição: a maioria dos alunos acessa a plataforma pelo celular) e o que precisa descobrir (dúvida: quem tem permissão para emitir a segunda via, só o aluno ou também o responsável financeiro?).

E — Elicitar

Coletar informações do mundo real sobre como o processo funciona hoje.

Elicitar é sair da opinião e ir para a realidade operacional. É a etapa em que você para de imaginar como o negócio funciona e começa a descobrir como ele de fato funciona. Cada conversa com um stakeholder nessa fase é uma oportunidade de encontrar uma regra de negócio que ninguém havia explicitado, um fluxo que acontece na prática mas não está em lugar nenhum, uma dor que o projeto precisa resolver mas que ainda não tinha nome.

Ferramentas e atividades:

- **Entrevistas e workshops com stakeholders:** conversas focadas em extrair como o processo funciona hoje, quais são as dores e o que precisa mudar
- **Análise de sistemas existentes:** investigação de como sistemas atuais se comportam, quais regras já estão implementadas e onde estão as lacunas
- **Fluxogramas do processo atual:** mapeamento visual do fluxo real de trabalho, antes de qualquer solução proposta
- **Cenários narrativos:** descrições em prosa de como um usuário real percorre uma situação de ponta a ponta, incluindo contexto, motivação e o que acontece quando algo dá errado. Diferente de um fluxo, o cenário conta uma história: quem é a pessoa, o que ela estava tentando fazer, o que encontrou pelo caminho. Esse formato revela requisitos que listas e diagramas escondem
- **Uso de IA:** com o contexto do projeto em mãos, usar modelos de linguagem para perguntar "quais regras de negócio e requisitos eu poderia estar esquecendo nesse cenário?". A IA não substitui o levantamento com stakeholders, mas funciona como um segundo par de olhos analítico que questiona o que ainda não foi pensado
- **Benchmarking:** análise de como produtos similares resolvem o mesmo problema, para identificar padrões, convenções e oportunidades
- **Prototipação exploratória:** esboços rápidos usados não para validar solução, mas para provocar reação e extrair informação de quem conhece o negócio
- **Diagrama de Caso de Uso:** representação visual da UML que descreve as funcionalidades de um sistema e as interações entre seus usuários (atores) e o software.

Resultado esperado:

- Necessidades e dores levantadas
- Lista inicial de requisitos funcionais e não funcionais
- Lista de Certezas, Suposições e Dúvidas atualizada

Exemplo: nas entrevistas com o time de suporte, o time descobre que alunos inadimplentes há mais de 60 dias não podem emitir segunda via sem autorização da coordenação financeira. Ninguém tinha mencionado essa restrição antes. Ela é uma regra de negócio que precisará guiar pelo menos dois requisitos funcionais novos. Um cenário narrativo escrito durante essa etapa descreve a jornada de uma aluna que tenta pagar uma parcela atrasada às 23h, no celular, e encontra o sistema bloqueado sem nenhuma explicação. Esse cenário sozinho revelou três requisitos de comunicação que não estavam em nenhum fluxo.

R — Refinar

Transformar informação bruta em estrutura.

Depois de elicitar, você tem muito material. Anotações de entrevistas, fluxos rabiscados, requisitos jogados em lista sem categoria, dúvidas que viraram certezas e certezas que viraram dúvidas. Refinar é a etapa mais analítica do processo. É aqui que você organiza, classifica, questiona e, principalmente, explicita as regras de negócio que estavam escondidas nas conversas.

Duas ferramentas são especialmente úteis nessa etapa. O **Checklist DIREC** ajuda a garantir que cada componente foi entendido de ponta a ponta, passando por Dados, Interações, Regras, Estados e Contexto. A **técnica "E se?"** força o fluxo feliz a mostrar suas bordas: cada "e se?" sem resposta é um requisito escondido esperando para aparecer no meio do desenvolvimento.

Ferramentas e atividades:

- **Checklist DIREC:** cinco lentes de análise aplicadas a cada componente: Dados (quais informações existem e de onde vêm), Interações (o que o usuário pode fazer e qual o feedback), Regras (o que é válido, o que bloqueia, quais são as exceções), Estados (em que situações o componente pode estar) e Contexto (muda por perfil, tempo ou integração?)
- **Técnica "E se?":** perguntas sistemáticas sobre o que acontece fora do fluxo principal. Cada "e se?" sem resposta é um requisito que vai aparecer mais

tarde, no pior momento possível

- **Demos do produto com o time:** apresentações do que foi construído até agora para provocar novas perguntas e identificar inconsistências entre o entendimento do time e a realidade do sistema
- **Categorização de requisitos:** organização dos requisitos levantados por tipo (funcional, não funcional) e por área, criando a estrutura inicial do backlog

Resultado esperado:

- Lista de requisitos organizada e categorizada
- Estrutura inicial de backlog
- Lista de Certezas, Suposições e Dúvidas revisada

Exemplo: ao passar o Checklist DIREC pelo componente de emissão de boleto, o time percebe que nenhuma das entrevistas respondeu o que acontece quando o gateway de pagamento está fora do ar no momento da geração. A técnica "E se?" revela mais três cenários não mapeados: e se o boleto gerado não chegar por e-mail? E se o aluno tentar gerar dois boletos para a mesma parcela? E se o boleto já tiver sido pago mas ainda aparecer como pendente?

V — Validar

Garantir que o entendimento está correto antes de seguir.

Validar é trocar "acho que entendi" por "estamos alinhados". É a etapa em que você leva o que foi refinado de volta para quem tem autoridade sobre o negócio e confirma, com exemplos e cenários concretos, que o entendimento está correto. Erros encontrados aqui custam uma reunião. Erros encontrados depois do desenvolvimento custam sprints inteiros.

Ferramentas e atividades:

- **Playback com stakeholders:** apresentação do entendimento consolidado para que os envolvidos confirmem, corrijam ou complementem o que foi levantado
- **Walkthrough de fluxos:** percurso guiado pelos fluxos mapeados, passo a passo, para identificar o que foi mal interpretado ou o que ficou faltando

- **Prototipação de baixa ou média fidelidade:** esboços funcionais o suficiente para provocar reação concreta de quem conhece o negócio, sem o custo de um protótipo acabado
- **Simulação de casos de uso reais:** encenação de situações reais com stakeholders para testar se os requisitos cobrem o que precisa ser coberto
- **Revisão de regras de negócio e exceções:** passagem sistemática pelas regras levantadas para confirmar que estão corretas, completas e com os responsáveis certos

Resultado esperado:

- Requisitos validados e regras de negócio confirmadas
- Ambiguidades resolvidas
- Ajustes feitos antes da especificação final

Exemplo: durante o walkthrough com a coordenação financeira, o time apresenta o fluxo completo incluindo a regra de bloqueio para inadimplentes acima de 60 dias. A coordenadora corrige: o prazo correto é 45 dias, não 60, e o desbloqueio pode ser feito pela própria secretaria, sem precisar escalar para a coordenação. Sem essa validação, o requisito teria sido especificado com uma regra errada.

E — Especificar

Traduzir tudo em algo acionável para design, desenvolvimento e QA.

A última etapa é onde o trabalho analítico se torna execução. Especificar é pegar tudo que foi contextualizado, elicitado, refinado e validado e transformar em histórias claras, com critérios testáveis, prontas para entrar no backlog. É aqui que o vocabulário das seções anteriores se aplica de vez: cada requisito funcional vira uma ou mais histórias de usuário, cada regra de negócio aparece como critério de aceite, e o time de desenvolvimento recebe o que precisa para construir sem precisar adivinhar.

Ferramentas e atividades:

- **Escrita de histórias de usuário:** tradução dos requisitos funcionais para o formato "Como [usuário], quero [ação], para que [benefício]", colocando a necessidade na perspectiva de quem vai usar

- **Definição de critérios de aceite:** condições testáveis que precisam ser verdadeiras para que uma história seja considerada concluída
- **BDD com estrutura *Given / When / Then*:** formato estruturado para escrever critérios de aceite como comportamentos observáveis: dado um contexto, quando uma ação acontece, então o sistema responde de determinada forma
- **User story mapping e fatiamento:** organização visual das histórias em sequência de uso para identificar dependências, prioridades e como quebrar histórias grandes em entregas menores
- **Definition of Ready:** checklist que define o mínimo que uma história precisa ter antes de entrar no desenvolvimento, evitando que itens incompletos avancem para o sprint

Resultado esperado:

- Histórias de usuário estruturadas
- Critérios de aceite claros e testáveis
- Backlog pronto para refinamento técnico
- Base consistente para design e engenharia

Exemplo: o requisito "emitir segunda via de boleto" vira a história "Como aluno com parcela em aberto, quero gerar uma segunda via do boleto para conseguir efetuar o pagamento mesmo após o vencimento." Os critérios de aceite cobrem o caminho feliz e os cenários descobertos no refinamento: dado que o aluno está inadimplente há mais de 45 dias, quando ele tenta gerar a segunda via, então o sistema exibe mensagem informando que a emissão está bloqueada e orienta a entrar em contato com a secretaria.

O CERVE não é um processo linear rígido. Na prática, você vai descobrir durante o refinamento que precisa voltar a elicitar. Vai encontrar na validação uma regra que exige um novo ciclo de refino. Isso é esperado e saudável. O que o framework garante é que nenhuma etapa seja simplesmente pulada porque "não havia tempo", porque o custo de pular sempre aparece, só aparece mais tarde e mais caro.

Na próxima seção, você vai ver esse processo funcionando em exemplos reais de projetos.